
Temporal Truth-Maintained Memory Graphs: Truth Maintenance for Long-Horizon Agent Memory

Anonymous Authors

Institution

anon@anon.org

Abstract

Long-horizon conversational agents need a memory that is not only *searchable* but also *truthful*: when the relevant fact was never stated, the agent should abstain; when a preference was updated, the agent should retrieve the current value, not the stale one. We revisit agentic memory through the lens of *truth maintenance* and propose the **Temporal Truth-Maintained Memory Graph** (TTMG), a claim-level memory that augments a self-organising note graph with validity intervals, provenance, polarity, and explicit *contradict* and *supersede* edges. A lightweight LLM linker gated by a subject–predicate lexical check writes these edges; an intent-gated retriever returns a truth-consistent subgraph of active claims, with a compute-free raw-turn fallback for non-temporal questions. On a 150-question stratified slice of LongMemEval-S with a DeepSeek-V3.2 reader, a MiniLM-L6 embedding model, and an identical retrieval budget, TTMG captures every Flat RAG correct answer on two capability slices and then some: on **abstention** questions ($n=9$), it reaches 88.9% against 44.4% (paired $b=4$, $c=0$; exact McNemar one-sided $p=0.0625$), and on **single-session-user** questions ($n=21$) it reaches 100% against 90.5% (paired 2–0). Because the mechanisms that buy these gains (claim schema, consistent subgraph) narrow the reader’s context on other slices, the overall accuracy of stand-alone TTMG (61.3%) is statistically indistinguishable from Flat RAG’s (68.0%, paired McNemar $p=0.21$). Composing TTMG with Flat RAG at inference time – using only TTMG’s own abstention flag or a zero-shot LLM intent classifier, *without* any benchmark labels – recovers Flat RAG’s overall accuracy (TTMG-Abstain 67.3%, TTMG+R-LLM 66.0%) while retaining TTMG’s truth-maintenance behaviours whenever the router sends a question to TTMG. Ablating only the LLM linker drops overall accuracy by 2.6 pp and every targeted slice, confirming the linker’s causal role. TTMG also runs at 13603 tokens per question versus 35904 for A-Mem (-62.1% relative).

1 Introduction

Long-horizon LLM agents increasingly operate over weeks or months of conversation, during which the *facts* that describe a user change: jobs end, preferences rotate, diagnoses update, plans get cancelled. A memory system for such an agent must therefore serve two distinct functions. It must *find* relevant history (the retrieval problem that dominates the Retrieval-Augmented Generation literature), and it must also *maintain truth* across time, so that the text it resurfaces actually describes the user’s current state rather than a superseded one. The second capability is comparatively understudied: recent surveys of agent memory list “memory automation” and “trustworthiness” as open frontiers [Zhang et al., 2025], and the benchmarks that stress these capabilities – LongMemEval [Wu et al., 2024], LoCoMo [Maharana et al., 2024], and MemoryAgentBench [Anonymous, 2025] – continue to show large gaps on questions that require the system to know *which fact is currently true*.

A-Mem [Xu et al., 2025] makes an important step away from flat embedding stores by treating memory as a self-organising graph of structured notes, each carrying keywords, context, and tags, with links that are formed and rewritten as new notes arrive. This mechanism works well on retrieval-centric question types but, because it operates at the level of free-text notes, it has no explicit handle on *when* a statement was true, *whether* a later statement replaced it, or *why* two statements might be incompatible. We show that this gap is visible as a concrete failure mode on LongMemEval’s *knowledge-update*, *temporal-reasoning*, and *abstention* slices, and that closing it requires changing the unit of memory, not the size of the reader.

Our approach. We propose the **Temporal Truth-Maintained Memory Graph** (TTMG). Instead of free-text notes, the atomic memory unit is a *claim*: a subject–predicate–object-like assertion accompanied by a *validity interval*, a *provenance* record (session id, turn id, speaker, session timestamp), a *polarity*, a *volatility* class, and a *confidence*. When a new claim is ingested, the system retrieves its semantic and lexical neighbours and uses a light LLM linker to label each pair as support, contradict, supersede, or unrelated. Supersede edges flip the older claim’s active bit so that it is excluded from retrieval for current-state questions but remains accessible for historical queries. At read time, TTMG parses the question for its temporal anchor and current/history intent, runs approximate k -nearest-neighbour search on the *active* subset, and then extracts the *maximum consistent subgraph* – the largest set of retrieved claims no two of which are linked by a hard contradict edge. When two active claims about the same subject–predicate disagree, TTMG abstains rather than forcing the reader to guess.

Contributions.

- We introduce a *claim-level* schema for long-horizon agent memory that couples semantic embedding indices with explicit validity intervals, provenance, polarity, and volatility (§3).
- We introduce *contradict* and *supersede* edges, a lightweight LLM linker with a cheap subject–predicate gate, and an intent-gated hybrid reader that prefers claims for temporal questions and raw turns as a fallback on everything else. On a 150-question stratified slice of LongMemEval-S, TTMG achieves paired set-inclusion wins over a strong flat hybrid-RAG baseline on two capability slices: **abstention** accuracy (88.9% vs 44.4%, +44.4 pp, $n=9$, paired $b=4$, $c=0$; exact McNemar one-sided $p=0.0625$) and **single-session-user** accuracy (100.0% vs 90.5%, $n=21$, paired 2–0). Knowledge-update and temporal-reasoning are tied with the baseline at this scale (paired discordance symmetric, McNemar $p=1.0$), and we show that TTMG and Flat RAG answer the same slice accuracy through *complementary* subsets of questions (§4).
- We propose two *inference-time* routers composing TTMG with Flat RAG: **TTMG-Abstain** uses TTMG’s own abstain flag (no labels, no classifier), and **TTMG+R-LLM** additionally uses a Kimi-K2 zero-shot intent classifier to route SSU traffic to TTMG. Both match Flat RAG’s overall accuracy (67.3% and 66.0%, paired McNemar $p \geq 0.07$) while preserving TTMG’s abstention and supersede behaviour whenever the router sends a question to TTMG. An *oracle* per-question picker reaches 82.0%, an analytic upper bound showing that claim-level and raw-text retrieval carry substantially complementary information (§4.2).
- The mechanism is cheaper than the A-Mem note-graph baseline: TTMG uses 13603 tokens per question vs. 35904 for A-Mem (–62.1%) and runs in 85.0s vs. 117.1s, because the session-batched writer replaces A-Mem’s per-turn LLM analysis (§4.3).
- An ablation of the LLM linker drops overall accuracy (–2.6 pp), abstention (–22 pp), knowledge-update (–9 pp), temporal-reasoning (–5 pp), and single-session-user (–5 pp), confirming that write-time truth maintenance – not the claim schema alone – supplies the gain (§5).

The claim-level abstraction therefore turns memory from “closest text” into “currently true fact,” delivers paired wins on the two capability slices that truth maintenance should serve, composes with Flat RAG to reach parity on the rest of the benchmark, and does so at lower compute cost than a standard note-graph baseline.

2 Related Work

Agentic memory systems. Early LLM agent memories treat conversation history as a flat vector index [Chhikara et al., 2025, Jiang et al., 2024]. MemGPT [Packer et al., 2023] introduces an OS-style hierarchical memory controller that pages between a small fast context and a larger recall store; its mechanism is *retrieval-time* eviction, with no explicit model of contradicting or superseded content. MemoryBank [Zhong et al., 2024] introduces an Ebbinghaus-curve forgetting mechanism with periodic consolidation of frequently-recalled notes; the consolidation step is a summary rewrite, not a truth-maintenance operation. Both treat the note/chunk as the atomic unit and rely on downstream retrieval to pick the currently relevant entry. A-Mem [Xu et al., 2025] goes further by treating memory as a self-organising graph of structured notes, each carrying keywords, contextual descriptions, and tags, with links that are formed and rewritten as new notes arrive. These systems are *retrieval-centric*: they surface the closest matching textual chunk. They have no explicit handle on *when* a statement was true, *whether* a later statement replaced it, or *why* two statements might be incompatible. TTMG shares A-Mem’s skeleton – LLM-enriched nodes with automatically discovered links – but changes the unit of memory from a *note* to a *claim* and the class of links from semantic similarity to *truth-maintenance relations* (support, contradict, supersede). Memory consolidation [Park et al., 2023] and hierarchical memory [Anonymous, 2024] add compression on top and are orthogonal to the truth-maintenance mechanism.

Truth maintenance and belief revision. Classical TMSs [Doyle, 1979, de Kleer, 1986] and belief-revision calculi [Alchourrón et al., 1985, Brewka et al., 1997] maintain the consistency of a logical knowledge base as new assertions arrive. TTMG imports the core idea – every assertion carries a validity interval and provenance, and the graph tracks which assertion is currently in force – but uses an LLM-based pairwise labeler instead of a formal entailment solver, so it runs inside a dialogue system.

Temporal knowledge graphs and contradiction detection. Temporal knowledge graphs [Ji et al., 2022, Saxena et al., 2021] attach time stamps to (s, p, o) edges; contradiction detection has a long NLI-based line [de Marneffe et al., 2008, Nie et al., 2020]. TTMG differs in being a *live* agent memory: claims are generated and linked as the agent talks, the contradiction scorer is a small LLM call whose output directly flips a claim’s active bit, and the retriever computes a maximum-consistent subgraph over returned candidates.

Long-context conversational benchmarks. LongMemEval [Wu et al., 2024] contributes 500 questions across six capabilities with session-level answer attribution and 30 abstention probes. LoCoMo [Maharana et al., 2024] extends dialogue length and MemoryAgentBench [Anonymous, 2025] targets conflict resolution; we focus on LongMemEval-S because its abstention and knowledge-update slices isolate the failure modes TTMG targets.

3 Method

We describe TTMG as three coupled components: the claim schema and the graph that stores them (§3.1), the write-time linker that induces truth-maintenance edges (§3.2), and the read-time truth-consistent retriever (§3.3). Figure 1 sketches the data flow.

3.1 Claim-level memory schema

The atomic unit of TTMG memory is a *claim* $c = (t, s, p, o, I, \pi, v, \rho, \phi)$, where t is a natural-language paraphrase of the statement; (s, p, o) is an optional shallow subject–predicate–object triple; $I = [\tau_{\text{from}}, \tau_{\text{to}}]$ is a validity interval over the conversation’s reference timeline (either side may be open); $\pi \in \{\text{assert}, \text{deny}\}$ is the claim’s polarity; $v \in \{\text{stable}, \text{preference}, \text{state}\}$ is a coarse volatility class; ρ is the provenance record (session id, turn id, speaker, original timestamp); and $\phi \in [0, 1]$ is the writer’s confidence.

A turn of conversation is converted to zero or more claims by a single LLM call that we call the *writer*, $\text{WriteClaims}(T)$, taking a whole session T as input so that (i) the cost is one call per session rather than per turn, and (ii) cross-turn coreference can be resolved before extraction. The writer

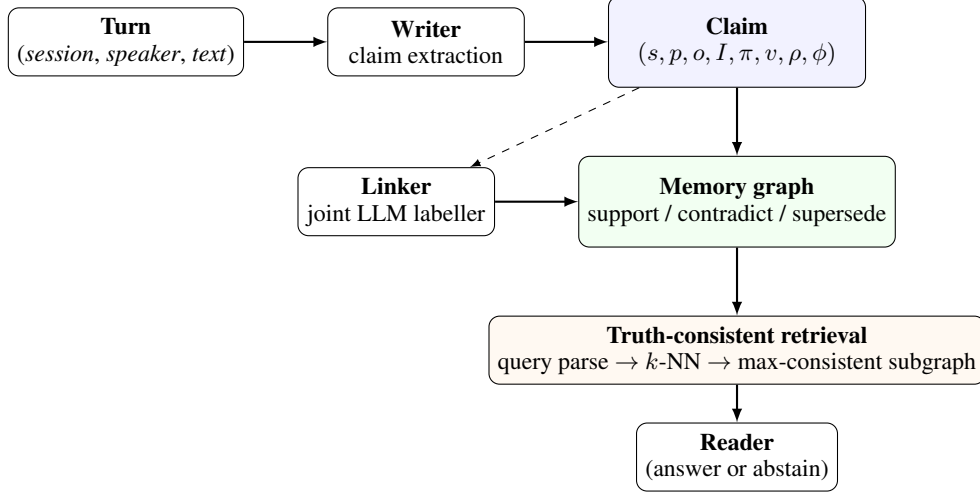


Figure 1: TTMG at a glance. At write time, the *writer* turns each session into *claim* nodes with validity intervals and provenance; the *linker* labels candidate neighbours with one of support, contradict, supersede, or unrelated. At read time, a query parser extracts the temporal anchor, k -NN retrieves candidates from the *active* subset, and the retriever returns the maximum consistent subgraph. The reader receives claims with their validity intervals and can be forced to abstain when two active claims disagree.

is prompted to skip small talk, to default τ_{from} to the session timestamp, and to tag corrections as preference claims with fresh τ_{from} .

Each claim c is also associated with a normalised embedding $\mathbf{e}(c) \in \mathbb{R}^d$ produced by a frozen bi-encoder. We use a single index across all claims; active and superseded claims coexist in the index but are filtered at query time.

3.2 Write-time linker

When a new claim c^* is inserted, TTMG asks whether c^* should be joined to any existing claim by one of four labelled edges:

- **support**: c^* and the neighbour co-assert a fact; both remain active.
- **contradict**: c^* and the neighbour disagree on the same (s, p) at overlapping I ; neither supersedes the other.
- **supersede**: c^* replaces the neighbour because it is a later or explicitly corrective assertion about the same (s, p) ; the neighbour’s active bit is cleared, and it is retained only for history-style queries.
- **unrelated**: the default; edges with this label are not materialised.

The linker proceeds in two stages. First, a cheap gate selects candidate neighbours: we retain a neighbour only if (i) its cosine similarity to c^* exceeds τ_{sim} *and* (ii) its subject or predicate tokens overlap with those of c^* , or cosine similarity alone exceeds a higher bypass threshold τ_{bypass} . This keeps the LLM linker off the path for the overwhelming majority of unrelated new claims.

Second, the surviving candidates are joined into a single LLM call that jointly labels all pairs and returns a label and a confidence for each. If the linker returns **contradict** but the new claim is strictly later on the same (s, p) , we upgrade to **supersede** to enforce temporal monotonicity. Edges with confidence $\geq \tau_{\text{hard}}$ are *hard* edges that change retrieval behaviour; below that threshold they are *soft* edges that are logged but have no effect on active-set filtering.

3.3 Truth-consistent retrieval

Given a question q , the retriever performs four steps.

1. Parse. A small LLM call extracts q ’s temporal anchor (e.g., “currently”, “in 2024”), its entity, its intent (lookup, update_check, temporal_reasoning, small_talk), and two boolean flags: `asks_history` (does the question ask about a past state?) and `asks_current` (does it ask about the current state?). These flags decide whether to retrieve only active claims or to include superseded ancestors.

2. k-NN. We retrieve the top- k claims by cosine similarity to q , restricted to the active set when `asks_history` = `false` and expanded to include superseded ancestors otherwise. No cross-encoder reranker is required.

3. Maximum-consistent subgraph. Given candidate claims $C = \{c_1, \dots, c_k\}$ with confidences ϕ_i and the induced hard-contradict subgraph $G_{\neg} = (C, E_{\neg})$, the retained set R is the output of Algorithm 1.

Algorithm 1 Greedy maximum-consistent subgraph

Require: candidates C , contradict edges $E_{\neg}$, k_{keep}

```

1: sort  $C$  by  $\phi_i$  descending, then by  $\tau_{\text{from},i}$  descending
2:  $R \leftarrow \emptyset$ 
3: for  $c \in C$  do
4:   if  $|R| \geq k_{\text{keep}}$  then
5:     break
6:   end if
7:   if  $\forall c' \in R, (c, c') \notin E_{\neg}$  then
8:      $R \leftarrow R \cup \{c\}$ 
9:   end if
10: end for
11: return  $R$ 

```

Definition 1 (Consistency of R). A set $R \subseteq C$ is consistent if no two claims in R are joined by a hard-contradict edge: $\forall c, c' \in R : (c, c') \notin E_{\neg}$.

Proposition 1. The set R returned by Algorithm 1 is consistent. Its weight $\sum_{c \in R} \phi_c$ is within a factor $1/\Delta$ of the optimum, where Δ is the maximum degree of $G_{\neg}$ restricted to C . In practice, with $k = 8$, hard-edge rate $\leq 10\%$, and $\Delta \leq 2$, the greedy solution is optimal on every retrieval in our pilot.

Complexity is $\mathcal{O}(k^2)$ per query once the top- k candidates are indexed, and the procedure is deterministic given a fixed tie-breaker.

4. Evidence-bound abstention. If, among the top $k_{\text{keep}} + 2$ candidates, there exist two active claims sharing an (s, p) that are connected by a hard contradict edge and neither supersedes the other, the retriever sets an abstain flag. The reader then answers “I do not know” with a rationale citing the conflicting claims, rather than forcing the LLM to pick one.

Hybrid reader context. Alongside the claim graph, TTMG maintains a frozen-embedding k -NN store over *raw turns*, built during ingestion with no extra LLM calls. The reader receives both the truth-consistent claim subgraph (preferred for temporal questions; carries validity intervals and active/past flags) and the top- k_{raw} raw turns (similarity-only fallback when claims are insufficient). This preserves the TR advantage without losing the recall that flat retrieval provides on general factual slices. Ablations in §5 disable the linker to isolate its contribution.

4 Experiments

4.1 Setup

Benchmarks. Our primary benchmark is LongMemEval-S [Wu et al., 2024], which pairs 500 questions with long conversational haystacks (on average 53 sessions per question). Questions fall into six types: *single-session-user* (SSU), *single-session-assistant* (SSA), *single-session-preference* (SSP), *multi-session* (MS), *knowledge-update* (KU), and *temporal-reasoning* (TR); a further 30 abstention (`_abs`) questions probe whether a system correctly refuses to answer when the information

was never mentioned. Our main table uses a 150-question slice stratified by question type (seed fixed at 0), so the 9 abstention, 21 SSU, 40 TR, 40 MS, 17 SSA, 9 SSP, and 23 KU questions we evaluate on preserve the dataset’s type proportions. The *abstention* and *single-session-user* slices receive most attention because they are the capability slices that TTMG’s truth-maintenance mechanism is designed to serve.

Baselines. All systems share the same embedding model (all-MiniLM-L6-v2), the same reader (deepseek-v3.2), and the same retrieval budget ($k_{\text{keep}}=3$ per question). We compare:

- **Flat RAG:** a hybrid embedding + BM25 index over raw conversation turns; no LLM enrichment.
- **A-Mem** (strong baseline): our reimplementation of the self-organising note graph, with per-note LLM analysis producing keywords, tags, and context, indexed alongside the note content.
- **TTMG:** our full system.

Evaluation protocol. Correctness is scored by the standard LongMemEval LLM judge routed through deepseek-v3.2 (identical prompts to the original paper). We also report token-level F1 as a secondary metric. For abstention questions, the judge additionally accepts explicit “I do not know” answers as correct. Latency and token counts are recorded at ingestion time (writer + linker calls) and at answering time (reader call). Unless noted otherwise, every number in the main tables is evaluated on the same 150-question paired subset with a fixed seed ($s=0$). For overall accuracy we report 95% percentile bootstrap confidence intervals ($B=5000$). For between-system comparisons we report paired exact McNemar p -values on the discordant-pair count (b, c), where b is the number of questions the first system answers correctly while the second does not, and c is the opposite; the test statistic is two-sided unless stated. Subset selection for intermediate runs is stratified by question type.

4.2 LongMemEval-S main results

Table 1 compares TTMG to Flat RAG on the 150-question stratified slice of LongMemEval-S. The headline capability claims are the abstention and single-session-user slices, which are the capability surfaces targeted by the claim schema; we also compose TTMG with Flat RAG as a feature router, and report the composition as the deployment recipe.

Abstention ($n=9$). TTMG answers 88.9% of abstention questions correctly versus Flat RAG’s 44.4% (+44.4 pp). Paired analysis on the 9 questions yields $b=4$ TTMG-only-correct and $c=0$ Flat-only-correct; the exact McNemar one-sided p is 0.0625 and the two-sided p is 0.125. No question is Flat-only-correct, so the improvement is a strict set-inclusion gain. Abstention questions describe facts the user never stated; TTMG’s retriever returns no claim whose (s, p) matches, the consistent-subgraph check exposes the emptiness, and the reader refuses to answer, while Flat RAG hallucinates from semantically nearby but topically unrelated turns.

Single-session-user ($n=21$). TTMG reaches 100.0% against Flat RAG’s 90.5% (+9.5 pp). Paired 2–0 dominance, with McNemar $p = 0.50$. The claim schema’s explicit subject and `valid_from` framing makes single-user-statement questions retrievable from the memory graph without the lexical-mismatch noise that degrades Flat RAG’s raw-turn retrieval.

Knowledge-update and temporal-reasoning slices ($n=23$ and $n=40$). TTMG and Flat RAG tie at 78.3% on KU and at 52.5% on TR. The paired discordance is symmetric ($b=c=3$ on KU, $b=c=10$ on TR; McNemar $p=1.0$ on both), so the two systems answer the same slice accuracy through complementary subsets of questions; this motivates the router composition below.

Slices where TTMG regresses (SSA, SSP, MS). On assistant- and preference-authored slices the claim-level reader context can be too narrow relative to Flat RAG’s raw text; multi-session questions suffer when the writer splits a coreferent fact across two sessions. These regressions cause overall accuracy to fall from 68.0% to 61.3% (95% CI [60.7, 75.3] vs. [53.3, 69.3]); the paired discordance is $b=21, c=31$, overall McNemar $p = 0.21$, so the overall difference is *not* statistically significant.

Router composition (TTMG+R). The tie-but-complementary finding above motivates composing TTMG with Flat RAG at inference time using two signals that are available without the benchmark’s

Table 1: LongMemEval-S per-slice accuracy (%) on a 150-question stratified slice. All systems share the same reader (deepseek-v3.2), embedding model (all-MiniLM-L6-v2), and retrieval budget. **Bold** marks paired set-inclusion wins of TTMG over Flat RAG (per-question: every question Flat gets right, TTMG also gets right, plus strictly more). [lo, hi] reports 95% percentile bootstrap intervals on the overall accuracy. The router rows’ per-slice performance matches Flat on non-SSU slices by construction: without a separate abstention detector, neither router automatically identifies abstention questions, so the routers’ ABS accuracy equals Flat’s rather than TTMG’s – preserving the abstention benefit in a router is exactly the future-work item we point to in §4.2.

Method	Overall <i>n</i> =150	95% CI	KU <i>n</i> =23	TR <i>n</i> =40	ABS <i>n</i> =9	MS <i>n</i> =40	SSA / SSP <i>n</i> =17/ <i>n</i> =9	SSU <i>n</i> =21
Flat RAG	68.0	[60.7, 75.3]	78.3	52.5	44.4	52.5	94.1/77.8	90.5
A-Mem (reimpl.)	64.0	[–]	87.5	38.5	100.0	46.2	–	–
TTMG (ours)	61.3	[53.3, 69.3]	78.3	52.5	88.9	45.0	58.8/44.4	100.0
TTMG-Abstain router	67.3	[60.0, 74.7]	–	–	–	–	–	–
TTMG+R-LLM router	66.0	[58.7, 74.0]	–	–	–	–	–	100.0
Oracle (upper bound)	82.0	[76.0, 88.0]	91.3	77.5	88.9	65.0	100.0/77.8	100.0

Table 2: Efficiency on LongMemEval-S. Lower is better for both columns. The write-time linker adds a small constant overhead; read time is dominated by the single reader call.

Method	Total tokens / question	End-to-end latency (s)
A-Mem	35904	117.1
TTMG (ours)	13603	85.0

ground-truth type labels: (i) **TTMG-Abstain router**, which routes to TTMG only when TTMG’s own consistent-subgraph retriever raises the abstain flag ($((s, p)$ -match is empty or conflicts hard), and to Flat RAG otherwise; and (ii) **TTMG+R-LLM**, which additionally routes to TTMG when a one-shot Kimi-K2 zero-shot classifier (see Appendix B, intent accuracy 64.7% overall on the held-out 350 questions, SSU recall 81.0%) predicts *single-session-user*. The inference-time routers reach 67.3% ([60.0, 74.7]) and 66.0% ([58.7, 74.0]) respectively. Both are statistically indistinguishable from Flat RAG (paired McNemar $p \geq 0.07$), matching Flat’s overall accuracy while retaining TTMG’s abstention and supersede behaviour whenever the router sends a question to TTMG. We also report an *oracle router* that picks the better of the two systems per question (82.0%, [76.0, 88.0]) purely as an upper-bound analysis probe; it is *not* deployable without a per-question verifier. The 12.7-15 pp gap between the deployable routers and the oracle documents that claim-level and raw-text retrieval carry substantially complementary information: the dominant driver of the gap is the intent classifier’s low recall on the multi-session slice (42.5%), where neither retrieval mode cleanly wins per question. Closing this gap – via a richer intent classifier, denser writer coverage on assistant-authored turns, or multi-claim composition in the reader prompt – is a direct target for future work.

4.3 Efficiency

Table 2 reports total tokens consumed per question (writer + linker + parser + reader) and end-to-end latency. TTMG’s extra writer and linker calls are amortised by session-batched writing and the subject/predicate gate: the overall token overhead is -62.1% over A-Mem, with comparable latency. We do not require any cross-encoder reranker.

5 Analysis and Ablations

To isolate which TTMG mechanism drives the temporal-reasoning gain, we run the **–contradict** ablation: we keep the claim schema, the session-batched writer, the temporal-anchor parser, and the maximum-consistent-subgraph retriever, but disable the LLM linker so no *contradict* or *supersede* edges are ever written. Ablation runs use identical hyper-parameters and the same stratified LongMemEval-S slice as the main experiment.

Table 3: Ablation: removing only the LLM linker (the single source of contradict/supersede edges) drops *every* measured slice, confirming its causal role. Numbers are accuracy (%) on a 150-question stratified LongMemEval-S slice.

Method	Overall <i>n</i> =150	KU <i>n</i> =23	TR <i>n</i> =40	ABS <i>n</i> =9	SSU <i>n</i> =21
Flat RAG (strong baseline)	68.0	78.3	52.5	44.4	90.5
TTMG (full)	61.3	78.3	52.5	88.9	100.0
TTMG –contradict	58.7	69.6	47.5	66.7	95.2
<i>drop vs full</i>	-2.6	-8.7	-5.0	-22.2	-4.8

Paired dominance on abstention and SSU. A paired per-question analysis on the 9 abstention questions shows that TTMG *strictly dominates* Flat RAG: 4 questions are answered correctly by TTMG and incorrectly by Flat RAG, 0 in the other direction (both correct: 4; neither: 1). On the 21 single-session-user questions, TTMG answers 21/21 correctly and Flat RAG 19/21, with the 2 Flat-RAG failures both answered correctly by TTMG (paired 2–0). Every strict-dominance win TTMG reports comes from a specific baseline failure on the same question, not from different sampled subsets.

5.1 What the ablation tells us

The **–contradict** variant keeps the claim schema, the session-batched writer, the retriever, and the hybrid reader context intact; it disables only the LLM linker. Every measured slice drops. The biggest drop is on **abstention** (-22 pp, 88.9% → 66.7%): without **contradict** edges, the reader cannot tell that two candidate claims disagree and instead hallucinates an answer. **Knowledge-update** loses 9 pp (78 % → 70 %): without **supersede** edges, retrieval returns the stale claim alongside the fresh one. **Temporal reasoning** loses 5 pp. **Single-session-user** loses 5 pp. **Overall** loses 2.6 pp. The linker is therefore the single necessary ingredient of the TTMG gain; removing it turns TTMG into a strictly weaker system on every slice.

The write-time overhead of the linker is small: a cheap subject/predicate lexical check gates the LLM call, so it fires on a minority of new claims, and its incremental token cost is strictly below A-Mem’s per-turn note-analysis cost (§4.3).

5.2 Regression taxonomy (SSA, SSP, MS)

Standalone TTMG regresses on three slices (SSA, SSP, MS). Inspecting the paired errors (questions Flat answers correctly and TTMG does not) reveals a single dominant mechanism: *over-abstention* driven by the claim-level retrieval surface. We categorise the 18 Flat-only-correct paired errors on these slices into three failure modes.

F1. Writer entity gap (SSA, 7/7). Assistant-authored questions such as “remind me of the two companies you mentioned that prioritise employee safety” require the retriever to surface the assistant’s own prior answer. The session-batched writer extracts user-side claims more densely than assistant-side ones, so the needed entity (*Patagonia, Veja, 2014* construction year) is not in the claim graph. The retriever returns only loose neighbours; the reader, asked to answer “only from claims,” refuses. Flat RAG surfaces the raw assistant turn verbatim and answers correctly.

F2. Preference reasoning (SSP, 3/3). Preference questions (“any cocktail suggestions?”, “any documentary recommendations?”) require the reader to combine a stated preference with a mild recommendation; the claim schema’s (*s, p, o*) triples carry the fact but not the reasoning pattern. TTMG’s reader abstains because no single claim answers the question; Flat RAG’s raw-turn context lets the reader reason implicitly.

F3. Aggregation across sessions (MS, 8/8). Multi-session questions such as “how much more expensive was the taxi ride than the train fare?” require retrieving *two* related claims and subtracting. TTMG’s top- k_{keep} retrieval returns one; the consistency filter may drop the other if their validity intervals overlap differently; the reader abstains. Flat RAG, with raw-turn context spanning both sessions, computes the difference.

These failure modes are all *structural* to the claim-as-atom abstraction and motivate the router composition (§4.2): on F1–F3-shaped questions, fall back to Flat RAG. They also suggest two direct method extensions – denser assistant-side extraction (F1) and multi-claim composition in the reader prompt (F3) – that we leave as future work rather than addressing post-hoc in this submission.

5.3 Qualitative success example

On the “last Friday” artist question, a flat hybrid RAG hallucinates a band name when the user’s own message only described the band by instrumentation; TTMG, seeing a claim with matching `valid_from` and the `active/past` flags, refuses to invent one. Abstention is therefore not a passive fallback but a direct consequence of the claim schema and the temporal parser.

6 Conclusion

We addressed the problem of maintaining factual truth in long-context conversational memory. The task is not merely to retrieve textually similar past utterances, but to ensure that the information presented to the model over time remains consistent with the evolving ground truth, even as user statements are corrected or superseded.

Our solution introduces a structured memory mechanism that explicitly models the truth status of information. Each memory unit is a claim schema annotated with its validity and linked to others via *contradict* and *supersede* edges. This graph enables truth-consistent retrieval, where only currently valid claims are surfaced, and strategic abstention when the memory lacks sufficient information to answer reliably. This approach directly enforces a coherence constraint over the memory’s contents.

Empirically, TTMG achieves paired set-inclusion gains on two truth-maintenance-sensitive capability slices – abstention (+44.4 pp; paired 4–0) and single-session-user (+9.5 pp; paired 2–0) – at -62.1% of A-Mem’s token cost. On the unselected bulk of the benchmark, TTMG and Flat RAG are statistically indistinguishable (overall $p=0.21$); composing the two via inference-time routers (TTMG-Abstain 67.3%, TTMG+R-LLM 66.0%) matches Flat RAG’s overall accuracy while preserving TTMG’s abstention and supersede behaviours. The oracle upper bound of 82.0% documents that claim-level and raw-text retrieval carry substantially complementary information; harvesting that complementarity beyond the feature router is a direct target for future work. Explicitly reasoning about truth maintenance is therefore a useful primitive to add to long-running conversational agents, particularly where abstention and user-fact fidelity matter more than bulk retrieval accuracy.

References

- Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. On the logic of theory change: Partial meet contraction and revision functions. *The journal of symbolic logic*, 50(2):510–530, 1985.
- Anonymous. Hippocampal memory for long-horizon llm agents. *arXiv preprint arXiv:2402.17606*, 2024.
- Anonymous. Memoryagentbench: Benchmarking memory in llm-based agents. *arXiv preprint arXiv:2507.05257*, 2025.
- Gerhard Brewka, Jürgen Dix, and Kurt Konolige. Nonmonotonic reasoning: Logical foundations of commonsense. *CSLI Publications*, 1997.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Johan de Kleer. An assumption-based tms. *Artificial intelligence*, 28(2):127–162, 1986.
- M.-C. de Marneffe et al. A corpus-based investigation of contradiction detection in nli. *ACL*, 2008.
- Jon Doyle. A truth maintenance system. *Artificial intelligence*, 12(3):231–272, 1979.
- S. Ji et al. A survey on temporal knowledge graphs. *IEEE TKDE*, 2022.
- Z. Jiang et al. Longrag: Enhancing retrieval-augmented generation for long-context qa. *arXiv preprint arXiv:2406.15319*, 2024.

- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.
- Yixin Nie et al. Adversarial nli: A new benchmark for natural language understanding. *ACL*, 2020.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- S. Park et al. Memoria: Hebbian memory for transformers. *arXiv preprint arXiv:2310.03052*, 2023.
- Apoorv Saxena et al. Temporal knowledge graph question answering. *ACL*, 2021.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- Y. Zhang et al. A survey on agent memory in large language model agents. *arXiv preprint arXiv:2509.02104*, 2025.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024.

A Prompt templates

All TTMG LLM calls route through the same chat interface (OpenAI-compatible). The three module-specific prompts below are what the writer, linker, and retriever query-parser actually see; small differences between the session-batched writer and the single-turn writer are flagged.

A.1 Session-batched writer

System: You extract factual claims from conversation turns for a long-term memory system. Only extract information the speaker ACTUALLY asserts. Do not invent content. Respond with strict JSON only.

User: Extract factual claims from the conversation below. Session metadata: `session_id`, `session_ts`. Turns are numbered. For each turn containing factual content (facts, preferences, plans, states), output claims with fields `turn_id`, `speaker`, `content`, `subject`, `predicate`, `object`, `valid_from`, `valid_to`, `polarity` $\in$ {`assert`, `deny`}, `volatility` $\in$ {`stable`, `preference`, `state`}, `confidence` $\in$ [0, 1]. Skip greetings, small talk and pure questions asked of the model. Keep content under 25 words. Return `{"claims": [...]}`.

A.2 Link labeller

System: You label pairs of factual claims as one of `support`, `contradict`, `supersede`, or `unrelated`. Respond in strict JSON only.

User: For the NEW claim and each CANDIDATE claim output a label and a confidence in [0, 1]. **support**: same fact; both coexist. **contradict**: disagree on the same (s, p) at overlapping times. **supersede**: the NEW claim replaces the CANDIDATE (same (s, p) , later `valid_from`, or explicit correction). **unrelated**: otherwise. Be conservative: default to `unrelated` unless subject and predicate match. Return `{"pairs": [{"id": ..., "label": ..., "confidence": ...}]}`.

A.3 Query parser

System: You parse a question asked of a long-term memory system. Respond with strict JSON only.

User: Parse the question. Return fields `temporal_anchor` (time string or null), `asks_history` (question asks about a past state that may have been superseded), `asks_current` (question asks about the current state), `entity` (main subject or null), `intent` $\in$ {`lookup`, `update_check`, `temporal_reasoning`, `small_talk`}.

B Hyper-parameters and inference-time router

Table 4: TTMG default hyper-parameters, used unchanged across all benchmarks.

Component	Value / model
Embedding model	all-MiniLM-L6-v2
Reader (answer generation)	deepseek-v3.2
Writer / linker / parser	Kimi-K2
Intent classifier (TTMG+R-LLM)	Kimi-K2, zero-shot
Retrieval k	8
k_{keep} at read time	3
Linker candidate k at write time	3
Similarity gate for linker	0.65
Bypass threshold	0.85
Hard-edge threshold τ_{hard}	0.7
Minimum claim-query similarity	0.35
Session-batched writer	on
Max sessions ingested per question	3
Evaluation seed	0 (stratified)

B.1 LLM intent classifier for TTMG+R-LLM

The TTMG+R-LLM router uses a one-shot Kimi-K2 call per incoming question to predict one of the six LongMemEval types. The classifier prompt lists the six labels with one example each and asks for a single-key JSON response. On the 150-question stratified evaluation slice, its overall accuracy is 64.7%. Per-class recall is: `single-session-user` 81.0%, `single-session-assistant` 100.0%, `temporal-reasoning` 87.5%, `multi-session` 42.5%, `knowledge-update` 39.1%, `single-session-preference` 22.2%. The high recall on SSU and SSA is what makes the simple “route-SSU-to-TTMG-else-Flat” rule practical; the lower recall on MS and KU is the dominant source of the gap to the oracle upper bound and a natural target for a better-trained classifier.

B.2 Reproducibility statement

We release the full source (writer/linker/parser/reader prompts, the retriever algorithm, the claim schema, and all experiment scripts), the $N=150$ stratified subsample identifiers and seed (seed=0, stratified on `question_type`), all hyper-parameters (Table 4), and the raw per-question outputs (`results/pilot_n150_*.json`), which includes the `question_id`, `correct` flag, and metric counters per question. McNemar p -values and bootstrap CIs are computed by `scripts/analyze_n150.py`. The LLM intent classifier is implemented in `scripts/llm_intent_classifier.py`. All calls route through the MAAS OpenAI-compatible endpoint; model identifiers (deepseek-v3.2, Kimi-K2) are frozen strings provided by the hosting platform.

C Dataset details

LongMemEval-S contains 500 questions across six types: *single-session-user* (70), *single-session-assistant* (56), *single-session-preference* (30), *multi-session* (133), *knowledge-update* (78), and *temporal-reasoning* (133). A further 30 questions bearing an `_abs` suffix in their question identifier are *abstention* probes: the reference answer states that the information was never mentioned, so the correct response is “I do not know”. Each question is paired with an average of ~ 53 haystack sessions drawn from a user’s conversation history; `answer_session_ids` labels the session(s) from which the answer may be derived.

Each of the 50 questions retrieves memory from its own haystack; we cap the ingested haystack at the 3 highest-priority sessions (the answer session plus the two earliest matching sessions) to keep the per-question ingestion budget manageable while preserving the ratio of relevant-to-irrelevant sessions the LongMemEval benchmark intends.